

Universidad de Sevilla
Máster en Ingeniería de Electrónica, Robótica y Automática (MIERA)

Percepción en Automática y Robótica

Simulación de Coche Autónomo: Detección de Carriles y Señales

Grupo XI

Samuel Boleslaw Locoche
Víctor Javier Granero Gil
José Francisco López Ruiz

Curso 2025–2026

Índice

1. Introducción	3
2. Detección de carriles	3
3. Detección y clasificación de señales	3
4. Integración en simulación y resultados	3
4.1. Entorno de simulación: CARLA Simulator	4
4.2. Arquitectura del sistema	5
4.3. Infraestructura: Docker y contenedores	5
4.4. Nodos ROS2 del sistema	6
4.4.1. Nodo de detección de carriles (<code>lane_detection_node</code>)	6
4.4.2. Nodo de detección de señales (<code>sign_detection_node</code>)	6
4.4.3. Nodo de control del vehículo (<code>vehicle_control_node</code>)	6
4.4.4. Nodo generador de anotaciones (<code>annotation_generator_node</code>)	7
4.5. Generación de datasets sintéticos	8
4.6. Resultados	9
4.6.1. Rendimiento en tiempo real	9
4.6.2. Entorno de pruebas	9
4.6.3. Conducción autónoma en simulación	9
5. Conclusiones	10

Resumen

Este trabajo presenta el diseño e implementación de un sistema de percepción para un vehículo autónomo simulado en **CARLA Simulator**, integrado con **ROS2 Humble** mediante contenedores Docker. El sistema comprende tres módulos principales: detección de líneas de carril mediante una red de segmentación semántica VGG16-U-Net, clasificación de señales de tráfico a través de una CNN entrenada con PyTorch, y un sistema de integración que une ambos módulos con un controlador PID doble para la conducción autónoma. El código fuente, modelos entrenados, datasets y vídeos demostrativos se encuentran en el repositorio público de GitHub: <https://github.com/JoseLopez36/Navegacion-Deteccion-Senales>.

1. Introducción

La conducción autónoma es uno de los retos tecnológicos más ambiciosos de la última década. Para que un vehículo sea capaz de desplazarse sin intervención humana debe ser competente en varias etapas: percepción del entorno, planificación de trayectoria y actuación sobre los sistemas de control. Este trabajo aborda la etapa de *percepción*, responsable de extraer información útil del entorno a partir de los datos sensoriales.

El objetivo del proyecto es desarrollar un sistema de percepción y control básico para un vehículo autónomo simulado en **CARLA Simulator**, conectado a través de **ROS2 Humble**. El flujo de funcionamiento diseñado es el siguiente:

1. Adquisición de imágenes desde la cámara virtual de CARLA.
2. Detección de líneas de carril mediante un modelo de segmentación semántica (VGG16-U-Net con TensorFlow/Keras), y estimación del error lateral del vehículo.
3. Detección y reconocimiento de señales de tráfico mediante una CNN implementada en PyTorch.
4. Interpretación de las señales para adaptar la velocidad del vehículo.
5. Publicación de comandos de control hacia CARLA.

Todo el código fuente, modelos entrenados, datasets y material multimedia (imágenes y vídeo demostrativo) están disponibles en el repositorio público de GitHub del proyecto:

<https://github.com/JoseLopez36/Navegacion-Deteccion-Senales>

Estructura de la memoria

La memoria se organiza en tres bloques principales:

Sección 2: Detección de carriles. Describe el dataset empleado, la arquitectura VGG16-U-Net y los resultados del modelo de segmentación.

Sección 3: Detección y clasificación de señales. Explica el proceso de detección por imagen y el diseño y entrenamiento de la CNN clasificadora.

Sección 4: Integración en simulación y resultados. Desarrolla la arquitectura software completa en CARLA/ROS2, el control del vehículo y los resultados obtenidos en simulación.

2. Detección de carriles

3. Detección y clasificación de señales

4. Integración en simulación y resultados

Esta sección describe cómo los dos módulos de percepción anteriores se integran dentro de un sistema completo que funciona en simulación. Se presentan el entorno de simu-

lación, la arquitectura software basada en ROS2, los nodos que componen el sistema, el controlador del vehículo y los resultados obtenidos.

4.1. Entorno de simulación: CARLA Simulator

Para el desarrollo y validación del sistema se ha empleado *CARLA Simulator* (*Car Learning to Act*), un simulador de conducción autónoma de código abierto basado en el motor gráfico *Unreal Engine 4*. CARLA proporciona sensores virtuales de gran fidelidad que permiten probar algoritmos de percepción y control sin necesidad de un vehículo real.

El simulador se ejecuta en su versión *0.9.15* dentro de un contenedor Docker con acceso a la GPU de la máquina anfitriona. Las características más relevantes del entorno configurado son:

- Mapa: Town04, una autopista con señales de tráfico y carreteras con múltiples carriles.
- Vehículo (*ego-vehicle*): modelo basado en Tesla Model 3, controlado desde ROS2.
- Sensores disponibles:
 - Cámara RGB frontal (**rgb_front**): utilizada para detección de carriles y señales.
 - Cámara de segmentación semántica (**semantic_segmentation_front**): empleada para la generación automática de datasets con *ground truth* perfecto.
 - Medidor de velocidad (**speedometer**): velocidad actual del vehículo en m/s.
 - Cámara exterior (**rgb_view**): vista en tercera persona para supervisión visual.

La Figura 1 muestra las dos perspectivas principales disponibles en el simulador.



(a) Vista exterior (tercera persona).



(b) Cámara frontal del vehículo.

Figura 1: Perspectivas en CARLA Simulator. Se aprecia el ego-vehicle tipo Tesla y, en la vista frontal, una señal de límite de velocidad a 90 km/h.

4.2. Arquitectura del sistema

El sistema completo se articula en tres capas de software que se comunican entre sí a través de una red Docker en modo *host*:

1. CARLA Simulator: simulador en contenedor Docker independiente con acceso a la GPU.
2. CARLA-ROS2 Bridge: puente oficial que traduce los datos de los sensores de CARLA en tópicos ROS2 estándar y convierte los comandos de control ROS2 en actuaciones sobre el vehículo simulado.
3. Sistema de navegación y percepción: paquete ROS2 propio (`navegacion_deteccion_senales`) que aloja todos los nodos de detección, clasificación y control.

El flujo de información entre los nodos es el siguiente: CARLA publica imágenes en `/carla/ego_vehicle/rgb_front/image` y la velocidad en `/carla/ego_vehicle/speedometer`. El `lane_detection_node` procesa esas imágenes y publica el error lateral en `/lane_detection/lane_error`. El `sign_detection_node` publica el límite de velocidad detectado en `/sign_detection/speed_limit`. El `vehicle_control_node` consume ambos tópicos para calcular y publicar los comandos de control en `/carla/ego_vehicle/vehicle_control_cmd`. Por último, el `annotation_generator_node` agrega toda la información disponible para generar una capa de visualización en tiempo real sobre *Foxglove Studio*.

4.3. Infraestructura: Docker y contenedores

Todo el sistema se orquesta mediante **Docker Compose**, levantando tres servicios con un único comando desde la raíz del repositorio:

```
1 xhost +local:docker
2 docker compose -f tools/docker-compose.yaml up
```

La siguiente tabla describe cada uno de los servicios del stack:

Servicio	Contenedor	Función
<code>ros_bridge</code>	<code>carla_ros_bridge</code>	Bridge CARLA→ROS2
<code>navegacion</code>	<code>navegacion_deteccion_senales</code>	Lanzamiento del paquete de ROS2
<code>foxglove</code>	<code>foxglove_bridge</code>	WebSocket bridge para Foxglove Studio

La imagen Docker base es `nvidia/cuda:12.8.0-cudnn-runtime-ubuntu22.04`, sobre la que se instalan:

- ROS2 Humble Hawksbill (distribución LTS sobre Ubuntu 22.04).
- PyTorch con soporte CUDA 12.8 para inferencia del clasificador de señales en GPU.
- ONNX Runtime con soporte GPU, para el modelo de detección de carriles.

- Cliente Python de CARLA 0.9.15 y `carla-ros-bridge` compilado desde el código fuente.

El código fuente del paquete ROS2 se monta como volumen dentro del contenedor de navegación, lo que permite que cualquier cambio en el host se aplique de forma inmediata (junto con `colcon build --symlink-install`) sin necesidad de reconstruir la imagen.

4.4. Nodos ROS2 del sistema

El paquete `navegacion_deteccion_senales` implementa seis nodos ROS2 en Python que se lanzan mediante el fichero `run.launch.py`.

4.4.1. Nodo de detección de carriles (`lane_detection_node`)

Este nodo recibe cada *frame* de la cámara frontal, ejecuta la inferencia del modelo VGG16-U-Net exportado en formato ONNX y calcula el error lateral del vehículo respecto al centro del carril, expresado en metros. El error se publica en `/lane_detection/lane_error` (Float32) y el estado completo del carril —incluyendo las coordenadas de las líneas izquierda y derecha— se serializa como JSON en `/lane_detection/lane_state`.

4.4.2. Nodo de detección de señales (`sign_detection_node`)

El nodo de señales emplea dos entradas: la imagen RGB de la cámara frontal y la imagen de segmentación semántica del mismo punto de vista. La segmentación semántica identifica con precisión los píxeles correspondientes a señales de tráfico (clase de color amarillo en CARLA), obteniendo *bounding boxes* candidatas que se validan por tamaño (`min_sign_area: 150 px`). Sobre el recorte resultante se ejecuta el clasificador CNN, que determina la etiqueta de la señal (`speed_limit_30`, `speed_limit_60`, `speed_limit_90`) y el límite de velocidad en m/s.

4.4.3. Nodo de control del vehículo (`vehicle_control_node`)

Este nodo cierra el lazo de control: recibe las percepciones de los nodos anteriores y genera los comandos de actuación sobre el vehículo. Opera a 10 Hz e implementa dos controladores PID independientes.

PID de velocidad (control de crucero). Regula la velocidad del vehículo en torno a una referencia configurable, inicializada en 2.78 m/s (≈ 10 km/h). Cuando el nodo de señales detecta un límite de velocidad inferior, la referencia se reduce automáticamente. El error de control es la diferencia entre la velocidad objetivo y la velocidad real medida por el odómetro. La salida del PID se mapea a los canales *throttle* y *brake* normalizados en $[0, 1]$:

$$\begin{aligned}
e_v(t) &= v_{\text{ref}} - v_{\text{actual}} \\
u_v(t) &= K_{p,v} e_v + K_{i,v} \int e_v dt + K_{d,v} \dot{e}_v \\
\text{throttle} &= \max(0, \min(1, u_v)), \quad \text{brake} = \max(0, \min(1, -u_v))
\end{aligned}$$

PID de dirección (mantenimiento de carril). Ajusta el ángulo del volante para mantener el vehículo centrado en el carril. La entrada es el error lateral en metros. Para suavizar la respuesta ante el ruido de la máscara de segmentación, se aplica un filtro paso-bajo de primer orden con coeficiente $\alpha = 0,05$ antes de alimentar al PID:

$$e_{\text{filt}}[k] = \alpha e_{\text{raw}}[k] + (1 - \alpha) e_{\text{filt}}[k - 1] \quad (1)$$

La salida del PID, en radianes, se normaliza por el ángulo físico máximo del volante ($\theta_{\text{máx}} = 0,6 \text{ rad} \approx 34$) para obtener el valor de dirección normalizado de CARLA en $[-1, 1]$. La convención de signo establece que un error positivo (vehículo desplazado a la derecha del carril) genera un giro a la izquierda (dirección negativa).

Ambos PIDs incorporan anti-windup que limita el término integral al rango $[-10, 10]$. Los parámetros finales configurados en `params.yaml` se recogen en la Tabla 1.

Cuadro 1: Parámetros de los controladores PID (`params.yaml`).

Parámetro	Valor	Descripción
<code>control_rate</code>	10 Hz	Frecuencia del bucle de control
<code>target_speed</code>	2.78 m/s	Velocidad objetivo ($\approx 10 \text{ km/h}$)
<code>kp_throttle</code>	0.30	Ganancia proporcional de velocidad
<code>ki_throttle</code>	0.05	Ganancia integral de velocidad
<code>kp_steering</code>	0.20 rad/m	Ganancia proporcional de dirección
<code>ki_steering</code>	0.02 rad/(m·s)	Ganancia integral de dirección
<code>kd_steering</code>	0.10 rad·s/m	Ganancia derivativa de dirección
<code>max_steer_rad</code>	0.6 rad	Ángulo físico máximo del volante

4.4.4. Nodo generador de anotaciones (`annotation_generator_node`)

Este nodo actúa como capa de visualización del sistema. En cada *frame* de la cámara frontal, recoge el estado de todos los demás nodos y genera un mensaje `foxglove_msgs/ImageAnnotations` que Foxglove Studio superpone sobre la imagen en tiempo real. Las anotaciones incluyen:

- Polígono de carril: región semitransparente en verde que cubre el área entre las líneas izquierda y derecha del carril.
- Líneas de carril: línea izquierda en amarillo y derecha en cian, con grosor de 6 px.
- Vector de desviación: segmento del centro de la imagen al centro del carril estimado; verde si el error es menor de 30 px, rojo si es mayor.

- Bounding box de señal: rectángulo con esquinas decorativas y etiqueta de texto; naranja para límites de velocidad, rojo para señales de stop.
- HUD de telemetría: textos con el error lateral (m y px), la velocidad actual (km/h) y los valores de *throttle*, *brake* y *steer*.

La Figura 2 muestra la interfaz de Foxglove Studio.



Figura 2: Interfaz de Foxglove Studio durante una sesión de conducción autónoma.

4.5. Generación de datasets sintéticos

Una de las ventajas de trabajar con un simulador como CARLA es la posibilidad de generar datos etiquetados de forma automática. La cámara de segmentación semántica asigna a cada píxel una etiqueta de clase con precisión perfecta, ofreciendo un *ground truth* ideal para entrenar y validar los modelos de percepción sin ningún coste de anotación manual.

El paquete implementa dos nodos de recolección que operan en modo conducción manual (sin `vehicle_control_node`) a una cadencia de 5 Hz:

Dataset de carriles (`lane_dataset_node`). Detecta las marcas viales en la máscara semántica, las proyecta al espacio RGB y almacena en `dataset/lanes/` pares de imagen RGB y máscara binaria de *ground truth*.

Dataset de señales (`sign_dataset_node`). Localiza los píxeles de señales (color amarillo en CARLA), extrae bounding boxes y almacena en `dataset/signs/` las imágenes RGB junto con ficheros JSON de anotaciones.

Las composiciones Docker para la recolección están en `tools/docker-compose-lane-dataset.yaml` y `tools/docker-compose-sign-dataset.yaml`. El repositorio incluye también herramientas de visualización (`visualize_lanes_dataset.py`, `visualize_signs_dataset.py`) y scripts de anotación manual (`annotate_signs_dataset.py`, `label_signs_dataset.py`) para enriquecer el dataset sintético con etiquetas adicionales. Las animaciones del proceso de generación de datos (`Lanes_Dataset.gif` y `Signs_Dataset.gif`) están disponibles en el repositorio de GitHub.

4.6. Resultados

4.6.1. Rendimiento en tiempo real

La siguiente tabla recoge los tiempos de procesamiento medidos durante las sesiones de conducción autónoma en el simulador.

Módulo	Latencia media	Frecuencia efectiva
Detección de carriles	≈ 50 ms	≈ 20 fps
Detección de señales	≈ 13 ms	≈ 77 fps

Ambas latencias se sitúan bien por debajo del período del controlador (100 ms a 10 Hz), lo que garantiza que los comandos de actuación siempre disponen de percepciones actualizadas.

4.6.2. Entorno de pruebas

Las pruebas se han realizado sobre el siguiente entorno hardware y software:

- CPU: Intel Core i7-13620H.
- GPU: NVIDIA GeForce RTX 5070 para portátiles.
- Sistema operativo: Ubuntu 22.04 (contenedor Docker).
- Framework: ROS2 Humble Hawksbill.
- Simulador: CARLA 0.9.15.

4.6.3. Conducción autónoma en simulación

El sistema completo mantiene el vehículo dentro del carril de forma continuada a velocidades de hasta 30 km/h, adaptando la velocidad cuando detecta señales de límite, pero escaladas a un tercio (i.e. 90 km/h pasa a ser 30 km/h). El vídeo demostrativo del sistema funcionando en tiempo real (`media/Demo.mp4`) está disponible en el repositorio de GitHub:

<https://github.com/JoseLopez36/Navegacion-Deteccion-Senales>

5. Conclusiones

Este trabajo ha permitido diseñar e integrar un sistema de percepción completo para la conducción autónoma en simulación, demostrando que la combinación de modelos de *deep learning* con una arquitectura ROS2 modular es una solución eficaz y reproducible. Los resultados obtenidos confirman que el sistema opera en tiempo real con latencias bajas y es capaz de mantener el vehículo dentro del carril mientras respeta los límites de velocidad detectados.